ПРОФЕСИОНАЛНА ГИМНАЗИЯ ПО ТЕЛЕКОМУНИКАЦИИ

ДИПЛОМЕН ПРОЕКТ

ТЕМА:

„ИЗПОЛЗВАНЕ НА ИЗКУСТВЕН ИНТЕЛЕКТ ЗА АНАЛИЗ НА ПОТРЕБИТЕЛСКИ МНЕНИЯ В УЕБ СРЕДА“

Изготвил: **Теодор Даниел Василев**
Професия: Икономист-информатик
Специалност: Икономическа информатика

Ръководител: **Мартин Ветов**

София, 2026

СЪДЪРЖАНИЕ

Увод ……..……..… 3

Глава 1. Теоретични и технологични основи ……..…….. 6
1.1. Теория на информацията – математическият фундамент ……..…..… 6
1.2. Компютърна лингвистика – историческо развитие …… 7
1.3. Как компютърът „чете" текст – от бит до символ до смисъл …… 8
1.4. Предобработка на текст – фундаментални операции ……. 9
1.5. Представяне на текст като числов вектор – от BoW до Embeddings …….. 10
1.6. Класически методи на машинното обучение за текстова класификация ……..… 12
1.7. Невронни мрежи – от перцептрон до LSTM и специализирани архитектури за текст 13
1.8. Transformer архитектура – революцията във векторното пространство ……..….. 15
1.9. Многоезични модели и предизвикателства на "Low-Resource" езиците ……..….. 17
1.10. Историческо развитие на езика Java и JVM платформата ……..... 17
1.11. Spring Framework и Spring Boot – архитектура и принципи ……..… 18
1.12. PostgreSQL – архитектура на релационна база данни ……..….. 19
1.13. Ансамблови методи за текстова класификация ……..… 20
1.14. Сравнителен анализ на съществуващи решения и обосновка на хибридния подход …. 20

Глава 2. Проектиране и архитектура на системата ……..…… 21
2.1. Функционални и нефункционални изисквания ……..… 21
2.2. Общ преглед на архитектурата …… 22
2.3. Технологичен стек – обосновка ……..…… 22
2.4. Модел на данните …….…… 23
2.5. Система за сигурност …….……... 23
2.6. REST API архитектура …… 24
2.7. Фронтенд архитектура (Flutter) …… 24

Глава 3. Имплементация на алгоритмите за текстова модерация …… 25
3.1. Обща архитектура на модерационния pipeline …… 25
3.2. WordFilterService: Детерминистичен In-Memory филтър …….. 26
3.3. Anti-Bypass нормализация …… 27
3.4. SentimentAnalysisService – 6-фазен конвейер …….…… 27
3.5. Интеграция на AI чат система (AiChatService) …… 29
3.6. Цикъл на обратна връзка и подготовка за самообучение (The Feedback Loop) …….. 29

Глава 4. Тестване, резултати и приноси …… 30
4.1. Тестване на конвейера за текстова модерация …… 30
4.2. Тестване на AI чат и сигурност …… 32
4.3. Анализ на резултатите и ефективност …… 34
4.4. Приноси на дипломния проект …… 34
4.5. Ограничения и насоки за бъдещо развитие …… 35

Заключение …… 36

Списък на използваната литература …… 37

Приложения …… 38
Приложение 1. ER диаграма на модела на данните …… 38
Приложение 2. Архитектурна диаграма на системата …… 39
Приложение 3. Примерни конфигурационни файлове …… 41
Приложение 4. URL адрес на хранилището с изходен код …… 42

УВОД

Днес всеки човек с достъп до интернет може да публикува мнение, коментар или рецензия за броени секунди. Платформи като Facebook, Twitter (X), Reddit, YouTube и стотици по-малки форуми генерират стотици милиони текстови публикации дневно. Според данни на Statista от 2024 г. само в Twitter се публикуват над 500 милиона туита на ден, а в YouTube се качават над 500 часа видео всяка минута, всяко от които може да събере хиляди коментари. Този колосален обем от потребителско съдържание (User-Generated Content, UGC) прави ръчната модерация практически невъзможна – дори най-големите технологични компании с хиляди модератори не успяват да обработят и малка част от постъпващото съдържание навреме.

Проблемът не е само количествен. Нежеланото съдържание в интернет приема разнообразни форми: реч на омразата (hate speech), обидни и токсични коментари, тормоз (cyberbullying), спам, дезинформация и пропаганда. Последствията от недостатъчната модерация са реални и измерими – проучване на Pew Research Center от 2021 г. показва, че 41% от възрастните американци са били обект на онлайн тормоз, а 25% от тях описват преживяването като „тежко". В европейски контекст Регламентът за цифровите услуги (Digital Services Act, DSA) на Европейския съюз, влязъл в сила през 2024 г., вече задължава платформите да предприемат активни мерки за премахване на незаконно съдържание, като предвижда сериозни санкции при неизпълнение.

В този контекст изкуственият интелект се превръща не просто в удобен инструмент, а в необходимост. Автоматизираната текстова модерация чрез методи на машинното обучение и обработката на естествен език (Natural Language Processing, NLP) позволява обработка на огромни обеми текст в реално време, с постоянство и без умора, които човешкият модератор не може да осигури. Същевременно AI системите не са безгрешни – те могат да пропуснат контекстен сарказъм, да не разберат културно-специфични обиди или да бъдат заобиколени чрез умишлена обфускация на забранени думи. Ето защо съвременните решения за модерация почти винаги комбинират автоматичен AI анализ с човешка проверка – подход, известен като „human-in-the-loop".

Настоящият дипломен проект има за цел проектирането и реализацията на уеб-базирана система за автоматична филтрация на потребителски текстови мнения чрез изкуствен интелект. Системата използва многофазен pipeline за анализ на сентимент, ансамблов подход за максимална надеждност, механизъм за самообучение чрез обратна

връзка от модератори и интегриран AI чат асистент. Реализацията е направена с Java 21 и Spring Boot 3.4 за backend, Flutter за кросплатформен frontend и PostgreSQL 17 за съхранение на данните.

Конкретните задачи, които си поставя проектът, са:

1. Да се анализират съществуващите подходи и технологии за автоматична текстова модерация, включително техните предимства и ограничения.
2. Да се проектира архитектура на система, която комбинира rule-based филтрация с елементи на машинно обучение и NLP обработка.
3. Да се имплементира 6-фазен pipeline за анализ на сентимент с anti-bypass нормализация, способна да разпознава обфускирани опити за заобикаляне на филтрите.
4. Да се реализира механизъм за самообучение (feedback loop), при който решенията на човешките модератори обогатяват бъдещите класификации.
5. Да се интегрира AI чат асистент с три доставчика (HuggingFace, OpenNLP, комбиниран режим).
6. Да се тества системата с разнообразни сценарии и да се анализира ефективността на избрания подход.

Очакваните резултати включват работеща система, която демонстрира, че хибридният подход – комбинация от бързи rule-based филтри, NLP обработка, облачни AI модели и човешка проверка – може да осигури ефективна модерация на потребителски мнения с висок recall (т.е. минимизиране на пропуснатите нарушения) при приемливо ниво на false positives.

Документът е структуриран в четири глави.

Глава 1 представя теоретичните основи – от фундаменталните концепции на теорията на информацията и компютърната лингвистика, през еволюцията на машинното обучение за текст, до съвременните Transformer модели и многоезичния NLP.

Глава 2 описва проектирането и архитектурата на системата, включително избора на технологии и тяхната обосновка.

Глава 3 е посветена на имплементацията на конкретните алгоритми за текстова модерация.

Глава 4 обхваща тестването, резултатите и приносите на проекта.

ГЛАВА 1. ТЕОРЕТИЧНИ И ТЕХНОЛОГИЧНИ ОСНОВИ

1.1. Теория на информацията - математическият фундамент

Преди да разгледаме как компютрите обработват и класифицират текст, е необходимо да установим математическия фундамент, върху който е изградена цялата съвременна обработка на информация. Този фундамент е положен от Клод Шанон в неговата фундаментална статия „A Mathematical Theory of Communication", публикувана през 1948 г. в Bell System Technical Journal. Шанон за пръв път дефинира информацията като измерима, математическа величина, независима от съдържанието или значението на съобщението.

$$H(X) = -\Sigma_i p_i \cdot \log_2(p_i)$$

За обработката на текст ентропията има пряко значение. Ако разгледаме букви в български текст като случайна величина, ентропията ни казва колко „информация" носи средно всяка буква. Шанон оценява ентропията на английския език на приблизително 1.0-1.5 бита на символ (значително по-ниско от $\log_2(27) \approx 4.75$ бита при равномерно разпределение), което означава, че естественият език е силно предсказуем и съдържа много излишък (redundancy). Именно тази предсказуемост е това, което позволява на езиковите модели да „предвиждат" следващата дума.

$$H(p, q) = -\Sigma_i p(x_i) \cdot \log_2(q(x_i))$$

Разликата H(p, q) - H(p) се нарича дивергенция на Кулбак-Лайблер (KL дивергенция):

KL дивергенцията измерва колко „далеч" е разпределението Q от P. Тя винаги е неотрицателна ($D_{KL} \geq 0$, с равенство само когато P = Q) и не е симетрична ($D_{KL}(P||Q) \neq D_{KL}(Q||P)$), поради което не е метрика в строгия математически смисъл, но е изключително полезна в машинното обучение.

$$PP(W) = 2^{H(W)}$$

Още едно забележително емпирично наблюдение, фундаментално за обработката на текст, е Законът на Zipf (1935). Джордж Кингсли Zipf наблюдава, че в почти всеки естествен език честотата на n-тата най-честа дума е приблизително обратно пропорционална на нейния ранг:

където $s \approx 1$. Това означава, че най-честата дума в даден корпус се среща приблизително два пъти по-често от втората, три пъти по-често от третата и т.н. В практически план това води до много неравномерно разпределение: малък брой думи (стоп-думи като „the", „is", „и", „на") доминират в текста, докато огромният „дълъг опашка" (long tail) от редки думи се среща изключително рядко. Този закон има дълбоки последствия за NLP: той обяснява защо стоп-думите са толкова проблематични, защо TF-IDF работи (честите думи получават нисък IDF), и защо subword токенизацията е ефективна (честите думи остават цели, редките се разбиват на по-чести подчасти).

$$PMI(w_1, w_2) = \log_2(P(w_1, w_2) / (P(w_1) \cdot P(w_2)))$$

1.2. Компютърна лингвистика – историческо развитие

Началото може да се проследи до Алън Тюринг и неговата статия „Computing Machinery and Intelligence" от 1950 г., в която той предлага знаменития „тест на Тюринг" (Imitation Game): ако машина може да проведе текстов разговор, неразличим от този на човек, тя може да се счита за „мислеща". Макар тестът на Тюринг да е философска конструкция, а не инженерна спецификация, той поставя фундаменталния въпрос: може ли машина да „разбере" текст? Този въпрос остава актуален и до днес.

Паралелно с инженерните усилия се развива и теоретичната лингвистика. Ноам Чомски, вероятно най-влиятелният лингвист на XX век, публикува „Syntactic Structures" през 1957 г. и формулира теорията за генеративните граматики. Чомски предлага йерархия на формалните граматики (днес известна като йерархия на Чомски), която класифицира езиците по тяхната изчислителна сложност:

Чомски аргументира, че естествените езици не са контекстно-свободни (макар CFG да са полезна апроксимация) и въвежда концепцията за дълбинна структура (deep structure) и повърхностна структура (surface structure) на изречението, свързани чрез трансформации. Тази теория оказва огромно влияние върху ранния NLP: десетилетия наред изследователите се опитват да конструират формални граматики, които да „парсват" (разбират синтактично) естествен език. Подходът, известен като rule-based NLP, доминира от 1960-те до края на 1980-те.

Друг забележителен проект от тази ера е SHRDLU (Terry Winograd, 1972) – система, която може да „разбира" команди на английски език, но само в контекста на виртуален свят от геометрични блокове. SHRDLU демонстрира, че дълбокото разбиране на езика изисква разбиране на света – проблем, който остава нерешен и до днес.

Фредерик Йелинек от IBM Labs е цитиран с апокрифната фраза: „Всеки път, когато уволня лингвист, разпознаването на реч се подобрява." Макар точността на цитата да е спорна, той улавя духа на епохата: данните и статистиката се оказват по-мощни от ръчно конструираните правила. Този принцип е валиден и до днес – съвременните езикови модели (GPT, BERT) са обучени на терабайти текст, без нито едно ръчно зададено граматично правило.

1.3. Как компютърът „чете" текст – от бит до символ до смисъл

Първият широко приет стандарт за кодиране на текст е ASCII (American Standard Code for Information Interchange), приет през 1963 г. ASCII използва 7 бита за кодиране на 128 символа: латинските букви (главни и малки), цифрите, пунктуационните знаци и някои контролни символи. Например буквата „A" е число 65 (1000001 в двоична система), „a" е 97, „0" е 48, интервалът е 32.

Решението на този проблем е Unicode – универсален стандарт, започнат през 1991 г. от консорциум от технологични компании. Unicode присвоява уникален code point (номер) на всеки символ от всяка писмена система: U+0041 за „A", U+0410 за кирилското „А", U+4E16 за китайското „世". Към версия 15.1 (2023 г.) Unicode съдържа 149 813 символа, включително emoji (U+1F600 за усмихнато лице), математически символи, музикална нотация и дори мъртви писмени системи като египетските йероглифи.

Значението на кодирането за текстова модерация е пряко и практическо. В настоящия проект anti-bypass нормализацията трябва да обработва опити за заобикаляне на филтрите чрез Unicode трикове: заместване на латински букви с визуално идентични кирилски (кирилското „а" (U+0430) изглежда като латинското „a" (U+0061), но е различен символ), използване на специални Unicode символи за визуално разделяне на думи (zero-width spaces, combining characters) и други. Системата адресира конкретно кирилско-латинската хомоглифия, като заменя кирилските „а"→"a", „е"→"e", „о"→"o", „с"→"c", „р"→"p", „х"→"x" при нормализацията – критично важна стъпка за модерация в българска среда.

1. Символно ниво (Character-level): работа с отделни символи – разпознаване на писменост, нормализация, case folding.
2. Лексикално ниво (Token-level): разделяне на текста на токени (думи, подчасти на думи или символи) – токенизация.
3. Морфологично ниво: анализ на вътрешната структура на думите – стеминг, лематизация, POS tagging.

4. Синтактично ниво: анализ на граматическата структура на изреченията – parse trees, dependency parsing.
5. Семантично ниво: извличане на значение – sentiment analysis, named entity recognition, word sense disambiguation.
6. Прагматично ниво: разбиране на контекст, намерение, ирония, сарказъм – най-трудното и все още нерешено ниво.

1.4. Предобработка на текст – фундаментални операции

Токенизация е процесът на разделяне на непрекъснат текст на дискретни единици – токени. На пръв поглед това изглежда тривиално: просто раздели по интервали и пунктуация. На практика токенизацията е изненадващо сложен проблем.

За разделяне на текст на изречения (sentence tokenization) проблемите са свързани с двусмислието на точката: „Dr. Smith earned $3.5M in the U.S." съдържа четири точки, но нито една от тях не бележи край на изречение. Punkt tokenizer-ът (Kiss & Strunk, 2006) решава този проблем чрез unsupervised обучение: автоматично разпознава съкращения (Dr., Mr., U.S.) и разграничава точка-край-на-изречение от точка-съкращение.

На ниво подчаст от дума (subword tokenization) съвременните модели използват значително по-софистицирани подходи. Byte Pair Encoding (BPE), предложен за NLP от Sennrich et al. (2016), стартира с речник от отделни символи и итеративно слива най-честата двойка съседни токени. Например, ако „th" е най-честата двойка, тя се слива в един токен, след това може „the" да стане един токен и т.н. Процесът продължава до достигане на желания размер на речника (напр. 50 000 токена). BPE е елегантно решение на проблема с непознати думи (Out-of-Vocabulary, OOV): нова дума, невиждана при обучението, автоматично се разбива на по-малки, познати подчасти.

Нормализация на текст включва привеждане на текста в стандартна форма. Case folding (привеждане в малки букви) е най-простата стъпка, но дори тя има капани: в турски „I" в малки букви е „ı" (без точка), а „İ" е „i" (с точка) – различно от английски! Unicode нормализация (NFC, NFD, NFKC, NFKD) привежда Unicode символите в каноничен вид: буквата „é" може да бъде кодирана като един символ (U+00E9, precomposed) или като два (U+0065 + U+0301, decomposed) – визуално идентични, но различни за компютъра.

Стеминг (stemming) е процесът на отрязване на суфикси от думи за получаване на обща основа: „running", „runs", „ran" → „run". Алгоритъмът на Портър (Porter, 1980) е най-известният stemmer – петстъпков процес от правила за суфиксно отрязване. Например: стъпка 1 премахва множествено число и -ed/-ing окончания; стъпка 2 заменя двойни

суфикси (ATIONAL → ATE, TIONAL → TION); стъпка 3 обработва -ful, -ness, -alize и т.н. Проблемите на стеминга са двустранни: over-stemming (различни думи получават еднаква основа: „university" и „universe" → „univers") и under-stemming (свързани думи получават различни основи).

За български език съществуват специализирани ресурси: BulStem (Nakov, 2003) е stemmer, разработен за българска морфология, а BulTreeBank (Simov et al., 2004) е treebank (корпус с анотирани синтактични дървета) за български. Въпреки това, ресурсите за български NLP са значително по-ограничени в сравнение с английски – ситуация, характерна за повечето „low-resource" езици, която мотивира използването на многоезични модели (вж. секция 1.9).

В настоящия проект regex-ите играят критична роля за anti-bypass нормализацията. Например, шаблонът `f[\*\\@\$\.\-\_]?[uv][\*\\@\$\.\-\_]?[c]?[\*\\@\$\.\-\_]?[k]` разпознава обфускирани варианти на определена ругатня, където между буквите може да има символи за маскиране. Шаблонът `[^aeiouаеиоуъяю\s]{7,}` детектира дълги последователности от съгласни (включително кирилски гласни в изключението), които указват gibberish/безсмислен текст. Този подход – използване на регулярни изрази за pattern matching на обфускиран текст – е пример за практическо приложение на теорията на формалните езици в content moderation.

1.5. Представяне на текст като числов вектор – от BoW до Embeddings

Фундаменталният проблем на обработката на текст с компютър е следният: компютърът работи с числа, а текстът е поредица от символи с произволно дължина. Преди каквото и да е машинно обучение, текстът трябва да бъде превърнат в числов вектор с фиксирана размерност – процес, известен като feature extraction или text representation. Качеството на това представяне определя горната граница на качеството на класификацията: дори най-мощният алгоритъм не може да извлече информация, която не е кодирана в представянето.

Bag of Words (BoW) подобрява One-Hot, като представя цял документ (а не отделна дума) като вектор на честотите на думите в него. За документ d и речник V = {w_1, w_2, ..., w_|V|}, BoW векторът е [count(w_1, d), count(w_2, d), ..., count(w_|V|, d)]. Множество от документи образуват Document-Term матрица, където всеки ред е документ, всяка колона е дума. BoW е основата на много класически ML методи за текст, но страда от два ключови недостатъка: загуба на реда на думите („кучето хапа човека" и „човекът хапа кучето" имат идентично представяне) и еднаква тежест на всички думи (предлогът „на" и ключовата дума „обида" получават еднакво третиране).

$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$

Интуицията е: дума, която се среща често в конкретен документ, но рядко в останалите документи, е вероятно важна за този документ. Стоп-думите като „the" имат висок TF, но нисък IDF (срещат се навсякъде), така че TF-IDF-ът им е нисък. Специфични термини като „токсичен" имат по-нисък TF, но висок IDF. Този подход, формализиран от Salton & Buckley (1988) като Vector Space Model, е основа на Information Retrieval десетилетия наред и все още е полезен baseline.

$P(w_t \mid w_1, w_2, ..., w_{t-1}) \approx P(w_t \mid w_{t-n+1}, ..., w_{t-1})$

$P(w_i \mid w_{i-1}) = count(w_{i-1}, w_i) / count(w_{i-1})$ (за bigram модел)

Character n-grams (последователности от символи вместо думи) са особено полезни за задачи като езикова идентификация, author attribution и – критично за нашия проект – разпознаване на обфускирани думи. Думата „f*ck" с character trigrams е ["f*c", "*ck"], а „f_u_c_k" е ["f_u", "_u_", "u_c", "_c_", "c_k"], като и двете запазват характерни шаблони на оригиналната дума.

Преди word embeddings тази идея се реализира чрез ко-поява (co-occurrence) матрици, където елементът X_{ij} брои колко пъти дума i и дума j се срещат в рамките на определен прозорец. Точковата взаимна информация (PMI) премахва ефекта от базовата честота:

PPMI (Positive PMI) заменя отрицателните стойности с нула (отрицателните PMI са статистически ненадеждни при малки корпуси). Singular Value Decomposition (SVD) върху PPMI матрицата дава нискоразмерни dense вектори – подход, известен като Latent Semantic Analysis (LSA, Deerwester et al., 1990).

- CBOW (Continuous Bag of Words): предсказва целевата дума от контекстните думи. Входът е средното на embedding-ите на контекстните думи, изходът е вероятностно разпределение върху речника. По-бърз, по-добър за чести думи.
- Skip-gram: обратното – предсказва контекстните думи от целевата. По-бавен, по-добър за редки думи.

maximize $\Sigma \Sigma \log P(w_{t+j} \mid w_t)$ за $j \in [-c, c], j \neq 0$

$P(w_O \mid w_I) = \exp(v'_\{w_O\}^T \cdot v_\{w_I\}) / \Sigma_w \exp(v'_w^T \cdot v_\{w_I\})$

$\log \sigma(v'_{w_O}{}^T \cdot v_{w_I}) + \Sigma_{i=1}^k E[\log \sigma(-v'_{w_i}{}^T \cdot v_{w_I})]$

Забележителното свойство на Word2Vec е, че обучените вектори улавят семантични и синтактични релации като линейни аналогии: vec(„king") - vec(„man") + vec(„woman") ≈ vec(„queen"). Това е емпирично доказателство, че дистрибутивната семантика кодира значение в геометрични отношения между вектори.

$J = \Sigma_{i,j} f(X_{ij}) \cdot (w_i^T \cdot \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$

FastText (Bojanowski et al., 2017, Facebook AI Research) надгражда Word2Vec, като представя всяка дума като сума от нейните character n-gram вектори. Думата „where" с n=3 се разлага на: <wh, whe, her, ere, re> плюс цялата дума <where>. Векторът на думата е средното на n-gram векторите. Ключовото предимство е обработката на непознати думи (OOV): нова дума, невиждана при обучението, все пак получава смислен вектор от своите n-gram компоненти. Това е особено ценно за морфологично богати езици (фински, турски, български), където различните форми на една дума (напр. „хубав", „хубава", „хубави", „хубавите", „най-хубав") могат да споделят n-gram компоненти и съответно семантична близост.

1.6. Класически методи на машинното обучение за текстова класификация

Наивният Бейсов класификатор (Naive Bayes) е вероятно най-простият, но изненадващо ефективен алгоритъм за текстова класификация. Той се базира на теоремата на Бейс:

където C е класът (напр. POSITIVE, NEGATIVE), а D е документът (коментарът). „Наивното" предположение е, че думите в документа са условно независими дадено класът:

Това предположение е очевидно неправилно (думите не са независими: „New" и „York" са силно корелирани), но на практика работи забележително добре. Причината е, че за класификация не е нужно да оценим точно P(C|D), а само да намерим argmax_C P(C|D), а ранжирането се запазва дори при нарушено предположение за независимост.

$P(w_i \mid C) = (\text{count}(w_i, C) + \alpha) / (\Sigma_w \text{count}(w, C) + \alpha|V|)$

Multinomial Naive Bayes работи с word counts (всяка позиция в BoW вектора е честота), а Bernoulli Naive Bayes работи с binary indicators (1/0 за наличие/отсъствие на дума). За текстова класификация Multinomial обикновено е по-добър. Complement Naive

Bayes (Rennie et al., 2003) е вариант, проектиран за imbalanced datasets – вместо да оценява $P(D|C)$, оценява $P(D|\bar{C})$ (вероятността при комплемента на класа), което подобрява производителността при неравномерно разпределение на класовете (например, ако 95% от коментарите са неутрални и само 5% са токсични).

$z = w^T \cdot x + b$

$\hat{y} = \sigma(z) = 1 / (1 + e^{-z})$

Обучението минимизира Binary Cross-Entropy Loss:

чрез Gradient Descent: $w \leftarrow w - \alpha \cdot \partial L/\partial w$, където $\partial L/\partial w_j = \Sigma(\hat{y}_i - y_i) \cdot x_{ij}$. Регуларизацията предотвратява overfitting: L1 регуларизация ($||w||_1$) насърчава sparsity (много тежести стават точно нула – автоматична feature selection), L2 ($||w||_2^2$) ограничава magnitudes (weight decay), а Elastic Net комбинира двете.

$P(y = k \mid x) = \exp(w_k^T \cdot x) / \Sigma_j \exp(w_j^T \cdot x)$

maximize $\Sigma\alpha_i - ½\Sigma\Sigma\alpha_i\alpha_j y_i y_j x_i^T \cdot x_j$ при $\alpha_i \geq 0$, $\Sigma\alpha_i y_i = 0$

Random Forest е ансамбъл от decision trees, обучени върху bootstrap samples от данните. Всяко дърво взема решения на базата на splitting criteria: ентропията на Шанон $H(S) = -\Sigma p_i \cdot \log_2(p_i)$ или Gini Impurity $= 1 - \Sigma p_i^2$. Information Gain $IG(S, A) = H(S) - \Sigma(|S_v|/|S|) \cdot H(S_v)$ измерва колко даден feature A „пречиства" класовете. При random forest всяко дърво вижда само random subset от features ($\sqrt{d}$ features per split), което декорелира дърветата и намалява variance-а.

1.7. Невронни мрежи - от перцептрон до LSTM и специализирани архитектури за текст

Развитието на изкуствените невронни мрежи (Artificial Neural Networks - ANN) представлява един от най-успешните примери за биомиметично инженерство, при което сложни математически и изчислителни модели черпят директно вдъхновение от биологичните процеси в човешкия мозък. Фундаментът на тази парадигма е положен през 1943 г., когато неврофизиологът Уорън Маккълох и математикът Уолтър Питс предлагат първия формален модел на биологичен неврон. Техният модел демонстрира, че мрежа от логически прагови елементи е способна да изчисли всяка изчислима функция (постигайки еквивалентност с машината на Тюринг). Няколко години по-късно, през 1949 г., Доналд Хеб формулира правилото за синаптична пластичност (познато като правилото на Хеб:

"Neurons that fire together wire together"), което поставя невробиологичните основи на алгоритмите за обучение.

Въпреки първоначалния огромен ентусиазъм, през 1969 г. Марвин Мински и Сиймур Паперт публикуват книгата "Perceptrons", в която нанасят съкрушителен теоретичен удар върху модела. Те демонстрират математически, че еднослойният перцептрон е принципно неспособен да реши дори елементарния нелинеен проблем изключващо ИЛИ (XOR). Макар че авторите споменават, че добавянето на "скрити слоеве" (hidden layers) би могло да реши проблема, по това време не съществува изчислително ефективен алгоритъм за тяхното обучение. Тази публикация води до т.нар. първа "AI зима" – период на значително съкращаване на финансирането и интереса към невронните мрежи, който продължава повече от десетилетие.

$$\frac{\partial L}{\partial w_{\mathrm{ij}}} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_{\mathrm{ij}}}$$

Активационни функции и проблемът със затихващия градиент

Критичен компонент на невронната мрежа е нелинейната активационна функция, която позволява на модела да усвоява сложни зависимости. Ранните мрежи масово използват логистичната сигмоидна функция $\sigma(x) = \frac{1}{1+e^{-x}}$ или хиперболичния тангенс $tanh(x)$. Двете функции обаче страдат от фатален недостатък при дълбоки мрежи (с много слоеве): проблемът със затихващия градиент (vanishing gradient problem). Тъй като производната на сигмоидната функция достига максимум от 0.25, при многократното умножение на тези производни в процеса на обратно разпространение през много слоеве, градиентите експоненциално клонят към нула. Това прави на практика невъзможно обучението на слоевете близо до входа.

Оптимизатори: Математиката зад сходимостта

Обучението на дълбоки мрежи за обработка на естествен език е изчислително тежка задача за оптимизация в не-изпъкнали пространства (non-convex optimization). Класическият Стохастичен градиентен спускане (Stochastic Gradient Descent - SGD) често попада в локални минимуми или седлови точки (saddle points).

Математическата формулировка на Adam изчислява първия момент (средното) m_t и втория момент (нецентрираната дисперсия) v_t на градиентите:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\, g_t$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)\, {g_t}^2$$

За да коригира пристрастието (bias) към нулата в началните стъпки, Adam прилага корекция:

$$\hat{m}_t = \frac{m_t}{1 - {\beta_1}^t}, \quad \hat{v}_t = \frac{v_t}{1 - {\beta_2}^t}$$

Актуализацията на тежестите се извършва по правилото: $w = w - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$.

В съвременния NLP се използва предимно AdamW вариантът, който отделя тегловното затихване (weight decay) от градиентната актуализация, осигурявайки значително по-добра генерализация.

Въпреки своята теоретична елегантност, класическите (Vanilla) RNN мрежи са на практика неизползваеми за дълги текстове (над 10-20 думи) поради вече споменатия проблем със затихващия или експлодиращ градиент. Ако разгледаме производната на скритите състояния във времето, тя включва многократно умножение на матрицата на рекурентните тежести W_{hh}. Ако собствените стойности на тази матрица са по-малки от 1, градиентът затихва (моделът "забравя" началото на изречението); ако са по-големи от 1, градиентът експлодира (моделът става числено нестабилен).

Тъй като актуализацията на C_t включва само събиране (а не матрично умножение), градиентите могат да протичат почти непроменени през хиляди времеви стъпки. По-късно, през 2014 г., Cho et al. предлагат Gated Recurrent Unit (GRU) – олекотена версия на LSTM, която слива forget и input портовете в един "update gate", постигайки сходна производителност с по-малко параметри и по-бързо време за обучение. В модела за анализ на сентимент, който настоящата система за модерация имплементира (чрез интеграцията с HuggingFace модели), въпреки че съвременните архитектури са базирани на Transformer, принципите на последователната обработка и извличане на контекст са дълбоко вкоренени в уроците, научени от LSTM.

1.8. Transformer архитектура – революцията във векторното пространство

През 2017 година, екип от изследователи от Google Brain и Google Research (Vaswani et al., 2017) публикува статията "Attention Is All You Need", която предизвиква фундаментална смяна на парадигмата в целия сектор на изкуствения интелект. До този момент, държавният стандарт (state-of-the-art) в обработката на текст се доминираше от сложни рекурентни (LSTM) или конволюционни (CNN) мрежи с добавени механизми за внимание (attention). Vaswani et al. (2017) доказват радикално твърдение: рекурентността не

е необходима; механизмът на самовнимание (self-attention) сам по себе си е достатъчен за моделиране на изключително сложни езикови зависимости.

Механизмът на внимание (Attention) и Scaled Dot-Product

Концепцията за внимание не е изобретена през 2017 г. Тя е въведена първоначално от Bahdanau (2014) и Luong (2015) за системи за машинен превод с цел да позволи на декодера да "надниква" към различни части от входното изречение (encoder), вместо да се разчита на един компресиран вектор с фиксиран размер.

Всяка дума във входната последователност се умножава по три различни обучеми матрици на тежестите (W^Q, W^K, W^V), за да се получат съответно Q, K и V векторите. Вниманието се изчислява чрез Scaled Dot-Product формулата:

$$\mathrm{Attention}(\mathrm{Q}, \mathrm{K}, \mathrm{V}) = \mathrm{softmax}\left(\frac{\mathrm{QKT}}{\sqrt{\mathrm{d_k}}}\right)\mathrm{V}$$

Multi-Head Attention и Позиционно кодиране

Единичният механизъм за внимание не е достатъчен, тъй като една дума може едновременно да бъде синтактично свързана с глагола си и семантично свързана с предходно съществително. Затова Transformer въвежда Multi-Head Attention: Q, K и V векторите се проектират h пъти (напр. 8 или 12 "глави") в по-нискомерни пространства, вниманието се изчислява паралелно за всяка глава, след което резултатите се конкатенират и проектират обратно. Това позволява на модела да улавя различни типове релации едновременно (синтаксис, семантика, кореференция).

Ерата на BERT (Bidirectional Encoder Representations from Transformers)

Публикуван от екип на Google през 2018 г. (Devlin et al., 2018), BERT използва само Encoder частта на Transformer архитектурата. Иновацията тук не е в самата архитектура, а в метода на предварително обучение (pre-training). BERT въвежда Masked Language Modeling (MLM): 15% от думите във входния текст се маскират случайно (заменят се с токена `[MASK]`), и задачата на модела е да отгатне кои са били тези думи, гледайки както левия, така и десния контекст едновременно. Това прави BERT дълбоко двупосочен (bidirectional). Втората обучителна задача е Next Sentence Prediction (NSP) – предсказване дали изречение B логически следва изречение A.

В контекста на настоящата диплома, модерацията чрез HuggingFace API използва модели базирани именно на RoBERTa (оптимизиран вариант на BERT без NSP и с по-

динамично маскиране), тъй като те предоставят ненадминато качество (State-of-the-Art) за класификационни задачи като разпознаване на токсичност (Sentiment & Toxicity Analysis).

1.9. Многоезични модели и предизвикателства на "Low-Resource" езиците

Multilingual BERT (mBERT) и феноменът Zero-Shot Cross-Lingual Transfer

Първият мащабен опит за създаване на универсален многоезичен модел е Multilingual BERT (mBERT), обучен едновременно върху корпуси от Wikipedia на 104 различни езика. mBERT не използва изрични индикатори за това какъв език чете в момента – той разчита на споделен WordPiece речник от 110 000 токена.

XLM-RoBERTa: Разрушаване на езиковите бариери

Въпреки успеха си, mBERT страда от т.нар. "проклятие на многоезичието" (curse of multilinguality): при фиксиран размер на параметрите на невронната мрежа, добавянето на нови езици започва да влошава качеството за всеки отделен език (възниква състезание за капацитета на модела).

За да реши този проблем, екип от Facebook AI Research (Conneau et al., 2020) създава XLM-RoBERTa (XLM-R). Моделът е обучен върху 2.5 терабайта (TB) нефилтриран текст, извлечен от уеб пространството (CommonCrawl) за 100 езика. От особена важност за настоящия дипломен проект е, че българският език е силно представен в тренировъчния корпус на XLM-R. XLM-R използва SentencePiece токенизация с огромен речник от 250 000 токена, който се справя несравнимо по-добре с различни скриптове (кирилица срещу латиница), избягвайки фрагментацията на непознатите думи.

1.10. Историческо развитие на езика Java и JVM платформата

Следващите секции (1.10-1.12) преминават от теоретичните основи на NLP и машинното обучение към технологичните платформи, върху които е реализирана системата. Тези технологии са представени тук, тъй като разбирането на тяхната вътрешна архитектура е предпоставка за обосноваване на проектните решения в Глава 2.

Изборът на технологичен стек за сървърната част (backend) на една високо натоварена система за модерация не е тривиален. Проектът използва Java 21, чиято екосистема е резултат от над три десетилетия еволюция в компютърните науки.

През 1991 г. екип в Sun Microsystems, воден от Джеймс Гослинг (James Gosling), стартира "Green Project" с цел създаване на език за интелигентни потребителски устройства. Езикът, първоначално наречен Oak и по-късно преименуван на Java, е проектиран около

една революционна за времето си концепция: "Write Once, Run Anywhere" (WORA). За да постигне това, Java не се компилира до машинно-специфичен код, а до междинно представяне, наречено байткод (bytecode), което след това се изпълнява от виртуална машина – Java Virtual Machine (JVM).

Този "горещ" код се изпраща към C1 компилатора (Client compiler) за бърза компилация с базови оптимизации. Ако кодът продължава да бъде интензивно използван, той се предава на C2 компилатора (Server compiler). C2 извършва агресивни оптимизации на ниво машинни инструкции, включително спекулативно вмъкване (speculative inlining), разгръщане на цикли (loop unrolling) и анализ на изтичането (escape analysis), който може да премахне заделянето на обекти в хийпа (heap) и да ги алокира директно в стека (stack). Благодарение на C2, производителността на дълго работещо Java приложение често достига, а в някои случаи на динамично оптимизиране дори надминава, тази на статично компилиран C++ код.

Еволюцията на GC е впечатляваща: от спиращия приложението (stop-the-world) Serial GC, през паралелния Parallel GC, до конкурентния CMS (Concurrent Mark Sweep). От Java 9 насам стандартен е G1GC (Garbage-First), който разделя паметта на региони и предсказуемо управлява паузите. Най-новите иновации като ZGC (въведен в Java 11) и Shenandoah постигат паузи под 1 милисекунда (sub-millisecond), дори при масиви от памет с размер на терабайти, благодарение на конкурентно компактиране чрез оцветени указатели (colored pointers) и бариери при зареждане (load barriers). Това е критично за уеб API-та, където дългите GC паузи водят до непредсказуема латентност.

Традиционно, всяка нишка в Java (стандартен `java.lang.Thread`) е директно обвита (mapped 1:1) около нишка на операционната система (OS thread). OS нишките са "тежки": те изискват около 1MB памет за стек и отнемат милисекунди за превключване на контекста (context switching) от ядрото. Това означава, че един сървър не може да поддържа повече от няколко хиляди едновременни нишки, което води до модела "thread-per-request" да се задъхва при високи натоварвания (I/O-bound tasks).

1.11. Spring Framework и Spring Boot – архитектура и принципи

Inversion of Control (IoC) и Dependency Injection (DI)

Фундаментът на Spring е Inversion of Control (Инверсия на контрола) – архитектурен принцип, известен още като "Холивудския принцип" ("Не ни търсете, ние ще ви потърсим"). Вместо обектите сами да създават своите зависимости (напр. използвайки оператора `new`), един централизиран контейнер (ApplicationContext) управлява жизнения цикъл на обектите (наречени beans) и ги "инжектира" там, където са необходими.

В настоящия проект това се реализира чрез конструкторна инжекция (Constructor Injection). Например, `CommentService` не създава сам инстанция на `SentimentAnalysisService` и `CommentRepository`, а ги получава през конструктора си. Това прави кода изключително модулен, слабо свързан (loosely coupled) и лесен за unit тестване (тъй като зависимостите могат лесно да бъдат подменени с mock обекти).

Spring Boot и Convention over Configuration

Spring Boot издига парадигмата на Spring едно ниво по-високо чрез принципа "Конвенция пред Конфигурация". В миналото Spring приложенията изискваха десетки XML файлове за конфигурация на сървъри, бази данни и beans. Spring Boot използва "auto-configuration": той анализира зависимостите (jar файловете) в проекта и автоматично конфигурира разумни стойности по подразбиране. Той също така вгражда уеб сървър (Tomcat по подразбиране), така че приложението може да бъде стартирано просто като стандартна Java програма с метод `main()`, без нужда от инсталиране и конфигуриране на външен сървър.

1.12. PostgreSQL – архитектура на релационна база данни

Релационният модел и ACID свойствата

Базата данни стриктно следва релационния модел, предложен от Едгар Код през 1970 г., и гарантира ACID свойствата:

- Atomicity (Атомарност): Всяка транзакция се изпълнява изцяло или изобщо не се изпълнява.
- Consistency (Консистентност): Данните винаги преминават от едно валидно състояние в друго, спазвайки всички външни ключове (Foreign Keys) и ограничения (Constraints).
- Isolation (Изолация): Едновременното изпълнение на транзакции води до състояние, което би било достигнато, ако те бяха изпълнени последователно.
- Durability (Трайност): След потвърждаване на транзакцията, данните не се губят дори при срив на системата. Този аспект се осигурява от механизма WAL (Write-Ahead Logging). Преди дадена промяна да бъде записана в същинските файлове с данни, тя се записва в WAL лог на диска. При срив, PostgreSQL чете WAL файла и "превърта" (replay) промените, за да възстанови състоянието.

Индексиране: B-Tree и GIN

Бързото извличане на данни се подсигурява от B-Tree (Балансирани дървета) индекси, които гарантират логаритмична сложност $O(log\, n)$ за търсене, вмъкване и изтриване. За задачи, свързани с търсене в текст или полуструктурирани данни (JSONB), PostgreSQL предлага Generalized Inverted Index (GIN), който е оптимизиран за заявки, при

които търсеният елемент се съдържа вътре в стойността (напр. търсене на ключова дума в масив от тагове или текст).

1.13. Ансамблови методи за текстова класификация

В машинното обучение нито един модел не е перфектен. Всеки класификатор има своите "слепи петна". Идеята зад ансамбловите методи (Ensemble Methods) е проста: комбиниране на предсказанията на множество независими модели за постигане на по-висока точност и устойчивост, отколкото който и да е от тях би постигнал сам.

Подходи като Bagging (Bootstrap Aggregating, напр. Random Forest) обучават множество дълбоки дървета (които имат нисък bias, но висока дисперсия) върху произволни подмножества от данните и осредняват резултатите им, за да намалят дисперсията. Подходи като Boosting (напр. AdaBoost, XGBoost) обучават поредица от "слаби" модели (напр. плитки дървета, които имат висок bias, но ниска дисперсия), като всеки следващ модел се фокусира върху грешките на предходния, по този начин намалявайки общия bias.

Поради тази причина, настоящата система имплементира ансамблов подход, базиран на логическо "ИЛИ" (Union). Анализът се извършва едновременно от:

1. Rule-based модул (речници и Regex за обфускация).
2. NLP статистически анализ (характеристики на текста: повтарящи се букви, дълги съгласни последователности).
3. Дълбоки невронни мрежи (HuggingFace Transformers).

Ако поне един от тези модули маркира коментара като подозрителен с увереност над определен праг, коментарът се маркира със статус `PENDING` (Очакващ човешка проверка). Това максимизира Recall метриката, гарантирайки, че почти нищо вредно няма да "протече" през филтъра. Този подход, макар да жертва малко Precision (увеличавайки обема работа за модераторите), е единственият индустриално приемлив стандарт за поддържане на безопасна среда.

1.14. Сравнителен анализ на съществуващи решения и обосновка на хибридния подход

Проектирането на система за автоматична модерация изисква задълбочен анализ на съществуващите алтернативи на пазара. Към момента решенията се разделят на три

основни категории: комерсиални облачни API-та, специализирани SaaS платформи и open-source библиотеки.

Комерсиални облачни API-та (Perspective API, OpenAI Moderation):

Google Perspective API е може би най-известното решение, използващо дълбоки невронни мрежи за оценка на „токсичност“. Основните му предимства са огромният мащаб на данните, върху които е обучено, и високата му прецизност. Въпреки това, то страда от т.нар. „vendor lock-in“ – зависимост от външен доставчик. Освен това, Perspective често има проблеми с езици с по-малко ресурси (като българския) и не позволява добавяне на специфични за дадена общност правила без сложна допълнителна конфигурация. OpenAI Moderation API предлага подобна функционалност, но е оптимизирано за филтриране на входове към LLM модели, което не винаги съвпада с нуждите на една форумната модерация.

Open-source решения (Detoxify, PerspectiveJS):

Библиотеки като `detoxify` позволяват локално хостване на модели, което решава проблема с поверителността на данните. Те обаче изискват значителни изчислителни ресурси (GPU) за работа в реално време, което ги прави скъпи за поддръжка при малки до средни проекти.

Обосновка на избрания хибриден подход:

Настоящият проект залага на хибриден модел, който комбинира най-доброто от двата свята:

1. Локални филтри (Rule-based): Осигуряват мигновена реакция за очевидни нарушения и пълна независимост.
2. Облачен AI (Transformer-based): Предоставя дълбоко семантично разбиране, без да натоварва локалната инфраструктура с тежки изчисления.
3. Human-in-the-Loop: Чрез Feedback Loop-а системата не е „черна кутия“, а се адаптира към решенията на реалните модератори.

ГЛАВА 2. ПРОЕКТИРАНЕ И АРХИТЕКТУРА НА СИСТЕМАТА

2.1. Функционални и нефункционални изисквания

Функционални изисквания:

- Система за управление на потребители с три нива на достъп (User, Moderator, Admin).
- Автоматичен анализ на всеки публикуван коментар през 6-фазния pipeline.

- Възможност за ръчно одобряване или отхвърляне на коментари със статус PENDING.
- Интерфейс за конфигурация на AI праговете (Thresholds) в реално време.
- Интегриран AI чат за помощ на потребителите.

Нефункционални изисквания:

- Скалируемост: Благодарение на виртуалните нишки в Java 21, системата трябва да обслужва висок брой конкурентни заявки.
- Ниска латентност: Автоматичният анализ не трябва да добавя повече от 500ms към времето за публикуване.
- Сигурност: Защита на личните данни и прецизен контрол на достъпа.

2.2. Общ преглед на архитектурата

Системата следва класическия архитектурен модел „Клиент-Сървър“, разделен на три основни слоя (3-Tier Architecture):

1. Презентационен слой (Frontend): Flutter приложение, което комуникира със сървъра чрез REST API.
2. Слой на бизнес логиката (Backend): Spring Boot приложение, съдържащо услугите за анализ, сигурност и обработка на данни.
3. Слой за съхранение (Database): PostgreSQL база данни за персистентност на информацията.

2.3. Технологичен стек – обосновка

Изборът на технологии е продиктуван от необходимостта за индустриална стабилност и производителност:
- Java 21: Избрана заради въвеждането на Project Loom. Традиционните backend езици като Python (през Flask/FastAPI) често срещат трудности при управление на хиляди конкурентни I/O операции без сложни асинхронни конструкции. Java 21 позволява писането на прост, блокиращ код, който се изпълнява върху леки виртуални нишки.
- Spring Boot 3.4: Най-разпространената рамка за Java, предлагаща отлична интеграция със Spring Security и JPA.
- Flutter: Позволява генерирането на мобилно и уеб приложение от една кодова база, което спестява време за разработка и поддръжка.

2.4. Модел на данните

Основните същности в системата са:
- User: Съдържа информация за потребителя, хеширана парола и роли.
- Comment: Основният обект на модерация. Пази оригиналния текст, нормализирания текст, Confidence Score-а от AI и текущия статус.
- AiSettings: Singleton същност, която съхранява глобалните конфигурации за пайплайна.
- ModerationTrainingData: Специализирана таблица за Feedback Loop-а, която събира данни за разминаванията между AI и човешките решения.

Енумерации (Enum Types):

За предотвратяване на грешки в данните, статусите се пазят като строго типизирани енумерации:
- `Role`: `USER`, `MODERATOR`, `ADMIN`
- `CommentStatus`: `APPROVED` (публично видим), `PENDING` (скрит, очакващ модерация), `REJECTED` (скрит поради нарушение).
- `Sentiment`: `POSITIVE`, `NEUTRAL`, `NEGATIVE`, `TOXIC`.

2.5. Система за сигурност

В съвременните уеб платформи, където се съхраняват лични данни и потребителски мнения, сигурността е абсолютен приоритет. Предложената архитектура имплементира сигурност на множество нива.

Автентикация без състояние (Stateless JWT)

За поддържане на потребителските сесии системата използва JSON Web Tokens (JWT). Традиционната сесийна автентикация изисква сървърът да пази идентификатор на сесията в паметта (RAM) за всеки активен потребител, което затруднява хоризонталното мащабиране (добавяне на нови сървъри).

JWT е "stateless" (без състояние) технология. Токенът се състои от три части:
1. Header (Заглавие): Указва алгоритъма за подписване (в нашия случай мощният `HS512` - HMAC-SHA512).
2. Payload (Полезен товар): Съдържа claims – твърдения за потребителя (неговото ID, `username`, `role` и време на изтичане `exp`).
3. Signature (Подпис): Генерира се чрез криптиране на Header-а и Payload-а с таен ключ (Secret Key), известен само на сървъра.

Ролево-базиран контрол на достъпа (RBAC) и Spring Security FilterChain

В Spring Security сигурността се реализира чрез верига от филтри (SecurityFilterChain). Всяка HTTP заявка, преди да достигне до бизнес логиката, преминава през филтъра за JWT валидация (`JwtAuthenticationFilter`). Този филтър извлича ролята на потребителя от токена и създава обект `Authentication` в `SecurityContext`-а на Spring.

Авторизацията (определянето кой какво може да прави) се управлява декларативно на ниво методи чрез анотации като `@PreAuthorize("hasRole('ADMIN')")`. Ако обикновен потребител се опита да достъпи endpoint за промяна на AI настройките, Spring AOP проксито ще прехване извикването и незабавно ще върне `403 Forbidden` грешка, без методът изобщо да се изпълни.

2.6. REST API архитектура

Комуникацията между клиента и сървъра се базира на REST (Representational State Transfer) принципите. Всеки ресурс (Comment, User) има свой уникален URI (напр. `/api/comments`).

- Statelessness: Сървърът не съхранява информация за предходните заявки на клиента. Всяка заявка съдържа цялата необходима информация (JWT).
- Uniform Interface: Използват се стандартни HTTP методи: `GET` за четене, `POST` за създаване, `PUT` за пълна промяна и `PATCH` за частична промяна (например само промяна на статус на коментар).
- JSON Format: Данните се обменят в лек, четим за човека формат, който се поддържа нативно както от Java (чрез Jackson библиотеката), така и от Flutter (чрез `dart:convert`).

2.7. Фронтенд архитектура (Flutter)

Архитектурата на клиентското приложение е изградена около фундаменталната концепция на Flutter: "Всичко е Widget". Потребителският интерфейс е представен като дървовидна структура от вложени компоненти (Widget Tree) (вж. Приложение 1).

AuthGate и Защита на маршрутите

Навигацията в приложението е защитена чрез компонент, наречен `AuthGate`. Това е умен навигационен слой, който слуша състоянието на автентикацията. Ако потребителят няма валиден JWT токен в локалното си хранилище (SecureStorage), `AuthGate` автоматично рендира екрана за вход (LoginScreen). Ако потребителят е логнат, той го насочва към съответното табло за управление (Dashboard) според неговата роля

(UserHomeScreen, ModeratorDashboard или AdminDashboard). Този механизъм гарантира, че неауторизиран достъп до защитени екрани е архитектурно невъзможен на ниво клиент.

ApiService: Централизиран мрежов слой

Всяка комуникация с REST API-то преминава през сингълтон клас (Singleton) `ApiService`. Това решение носи няколко ключови ползи:

1. Конфигуриране на базов URL адрес за всички заявки на едно място.
2. Автоматично прикачване на JWT токена в HTTP хедърите за всяка защитена заявка.
3. Глобално прехващане на грешки. Например, ако сървърът върне `401 Unauthorized` (когато токенът на потребителя изтече), `ApiService` улавя този код глобално, автоматично изчиства локалното хранилище и форсира пренасочване към екрана за вход, осигурявайки плавно и сигурно потребителско преживяване.

ГЛАВА 3. ИМПЛЕМЕНТАЦИЯ НА АЛГОРИТМИТЕ ЗА ТЕКСТОВА МОДЕРАЦИЯ

Тази глава разглежда същинската реализация на алгоритмите, които съставляват сърцевината на дипломния проект. Системата за модерация не разчита на един-единствен подход (silver bullet), а използва комплексен, многослоен конвейер (pipeline), който филтрира текста от най-простото към най-сложното ниво на анализ. Този хибриден подход осигурява оптимален баланс между скорост, прецизност и устойчивост на злонамерени атаки.

3.1. Обща архитектура на модерационния pipeline

Блок-схема на потока на данните (вж. Приложение 2):

1. Потребителски вход (Input): Суровият текст постъпва в системата.

2. Anti-bypass нормализация (Pre-processing): Текстът преминава през агресивна нормализация за изчистване на опити за обфускация (напр. смесване на кирилица с латиница, leet-speak, повтарящи се символи).

3. Бърза детерминистична филтрация (Rule-based Fast Path): Нормализираният текст се проверява срещу заредените в паметта `O(1)` речници чрез `WordFilterService`. Ако бъде открито директно съвпадение с черния списък (Blacklist), процесът може да спре тук със статус `PENDING`.

4. Многофазен AI анализ (6-Phase Sentiment Analysis): Ако текстът премине бързия филтър, той се подава на `SentimentAnalysisService`. Този модул извършва 6 последователни фази

на анализ: от детекция на Gibberish (безсмислен текст), през локални Heuristics, до отдалечени заявки към HuggingFace Transformer модели. Всяка фаза генерира частичен резултат (Partial Confidence Score).

5. Агрегация и вземане на решение (Ensemble Decision): Окончателният резултат (Final Confidence) се сравнява с динамично конфигурираните прагове (Thresholds), зададени от администраторите в `AiSettings`.
6. Присвояване на статус: Въз основа на праговете, коментарът получава статус `APPROVED` (публикува се веднага) или `PENDING` (отива в модераторската опашка за ръчен преглед).

7. Модерация и обратна връзка (Feedback Loop): Човешкият модератор преглежда `PENDING` коментарите. Неговото решение (одобрение или отхвърляне) създава запис в `ModerationTrainingData`, който в бъдеще се използва за дообучаване на модела.

Ансамблов принцип (Logical OR Strategy):

Ключов архитектурен принцип тук е ансамбловата OR стратегия. За да се максимизира метриката Recall (минимизиране на пропуснатите обиди), системата възприема дефанзивна стратегия: ако поне един от етапите (WordFilter, Local Regex Heuristics или Cloud AI) маркира коментара като потенциално токсичен, цялостният резултат се вдига над прага за `PENDING`. По-безопасно е човешки модератор да прегледа един нормален коментар (False Positive), отколкото вреден коментар да стигне до публичната стена на приложението (False Negative).

3.2. WordFilterService: Детерминистичен In-Memory филтър

Преди текстът да бъде изпратен за сложен семантичен анализ, той се обработва от WordFilterService. Целта на този сервиз е да действа като "бърза писта" (fast path) за очевидни нарушения.

Архитектура и зареждане на данни:

Сервизът управлява четири отделни списъка с думи: `toxic_words.txt`, `negative_words.txt`, `positive_words.txt` и `sensitive_topics.txt`. Зареждането им става по време на инициализацията на Spring контекста чрез метода, анотиран с `@PostConstruct`. Файловете се четат от файловата система и се зареждат в паметта.

Механизъм за презареждане по време на изпълнение (Atomic Runtime Reload):

Едно от основните изисквания към модерационната система е способността ѝ да реагира на нови тенденции (напр. нов сленг) без рестартиране на сървъра (zero downtime).

`WordFilterService` имплементира метод `reloadDictionaries()`, който създава нови инстанции на хеш таблиците в паметта, чете файловете и накрая подменя референциите към старите таблици с новите. Тъй като пренасочването на референция в Java е атомарна операция, това гарантира, че нито една обработваща се в момента заявка няма да изпита блокиране (locking) или да прочете непълни данни (dirty read).

3.3. Anti-Bypass нормализация

1. Кирилско-латинска хомоглифия (Lookalike characters):

Много букви в латиницата и кирилицата изглеждат визуално идентично (напр. кирилско 'а' `U+0430` и латинско 'a' `U+0061`), но компютърът ги разпознава като напълно различни числа в Unicode таблицата. Ругатня, написана на кирилица, в която една буква е заменена с латинския ѝ еквивалент, няма да бъде хваната от кирилския черен списък. За да предотврати това, нормализаторът извършва стриктен mapping, конвертирайки всички кирилски "двойници" към техните латински еквиваленти (или обратно) в унифициран формат преди сравнението.

2. Leet Speak и Символно маскиране:

Техниката "Leet speak" (напр. замяна на 'a' с '@', на 'e' с '3') се обработва чрез предварително дефинирани речници за замяна. Нормализаторът транслитерира тези символи обратно до оригиналните им буквени стойности. Допълнително се премахват разделители като точки, тирета и звездички вътре в самите думи (напр. `f.u.c.k` се нормализира до `fuck`).

3. Word Boundary Detection (Детекция на граници на думи):

При търсене на забранени думи възниква проблемът "Scunthorpe problem" – фалшиво положително блокиране на невинна дума, защото съдържа в себе си забранена подниз (например блокиране на английската дума "assassin" или българската "курс"). За да се реши това, системата използва регулярни изрази с Negative Lookbehind и Negative Lookahead: `(?<![a-zA-Z])word(?![a-zA-Z])`. Този шаблон гарантира, че думата "word" ще бъде засечена само ако преди и след нея няма други букви (т.е. тя е самостоятелна дума, а не част от друга дума).

3.4. SentimentAnalysisService - 6-фазен конвейер

Фаза 1: Детекция на безсмислен текст (Gibberish / Spam Detection)

Модерацията не се свежда само до реч на омразата; спамът и безсмислените коментари ("asdfghjkl") също влошават качеството на платформата. Системата използва евристичен алгоритъм:

- Consonant Sequences: Търси дълги последователности от съгласни букви (напр. над 5 съгласни подред), използвайки regex `[^aeiouаеиоуъяю\s]{5,}`. В естествените езици подобни струпвания са изключително редки.

- Character Diversity: Изчислява отношението на уникалните символи към общата дължина на стринга. Много ниско разнообразие (напр. "aaaaaaaaaa") индикира спам.
- Word Length: Детектира неестествено дълги думи (над 30 символа без интервал).

Ако се засече Gibberish, `Confidence Score` автоматично скача (напр. добавя се 0.4 към токсичността), тъй като такъв коментар трябва да бъде проверен или скрит.

Фаза 2: Локален речников анализ (Local Keyword Matching)

Това е интеграцията с `WordFilterService`. Текстът се токенизира (разделя на думи) и всяка дума се търси в `O(1)` хеш таблиците за токсични думи. За всяко открито съвпадение, към резултата се добавя тежест. Този етап е светкавично бърз и улавя значителна част от елементарните нарушения, при които токсичността е изразена чрез конкретни забранени думи.

Фаза 3: Научени негативни модели (Learned Negative Patterns)

Тази фаза проверява текста срещу модели, научени от предишни модераторски решения. Системата извлича токени от таблицата ModerationTrainingData (записи, създадени при ръчна модерация от Feedback Loop-а в секция 3.6) и ги сравнява с нормализирания текст. Ако бъде открито съвпадение с научен негативен модел, коментарът получава сентимент NEGATIVE с Confidence 0.85. Тази фаза позволява на системата да се адаптира към нови форми на токсичност, които не са предвидени в първоначалните речници – например нов сленг или специфични за общността обиди, които модераторите вече са отхвърлили ръчно.

Фаза 4: NLP анализ чрез HuggingFace Transformer (Cloud AI)

Ако предишните фази не са взели категорично решение (или за потвърждаване на съмнения), текстът се изпраща към безплатния HuggingFace Inference API (или локално хостван контейнер). Този етап използва предварително обучен модел (напр. много-езичен `XLM-RoBERTa`, фино настроен за sentiment/toxicity анализ).

Системата изпраща JSON заявка. Transformer моделът извършва семантичен анализ, който улавя контекст, който регулярните изрази не могат. (Например, "Ти си пълен боклук" няма забранени думи, но семантиката е изключително токсична). Отговорът от модела се

парсва, и ако върнатият етикет е `NEGATIVE` или `TOXIC`, съответната вероятност (която моделът връща) директно се добавя към общия `Confidence Score` на пайплайна. Извикването е имплементирано асинхронно, използвайки възможностите на Java 21 Virtual Threads, за да не се блокира сървърът при латентност от страна на API-то.

Фаза 5: Балансиране на сентимента

Модерацията трябва да отчита и позитивния контекст. Системата проверява за позитивни думи. Ако текстът е силно позитивен, това генерира "отрицателна тежест", която намалява общия `Confidence Score` за токсичност. Това е важно за избягване на фалшиви позитиви, например в коментари като "Това видео е жестоко!".

3.5. Интеграция на AI чат система (AiChatService)

Освен пасивната модерация, платформата предлага интерактивен AI чат асистент, реализиран чрез `AiChatService`. Тази архитектура демонстрира как различни типове AI (класификационни и генеративни) могат да съжителстват в една система.

Този хибриден подход към чатботовете решава класическия проблем с халюцинациите (hallucinations) при LLM моделите: когато става въпрос за строги правила на платформата (напр. бан политики), ние не искаме LLM да "си измисля" отговори, затова разчитаме на класическия MaxEnt (OpenNLP) за насочване към статични отговори. За общ разговор разчитаме на LLM.

3.6. Цикъл на обратна връзка и подготовка за самообучение (The Feedback Loop)

Процес на обогатяване:

1. Когато AI системата вземе решение, тя го записва в базата данни (напр. `Score = 0.85`, Статус `PENDING`).

2. Човешкият модератор отваря Dashboard-а и вижда този коментар. Ако той маркира коментара като `APPROVED` (Одобрен), това означава, че AI системата е допуснала False Positive (сметнала е невинния коментар за токсичен). Ако го отхвърли `REJECTED`, това е True Positive.

3. При натискане на бутона за одобрение/отхвърляне, `CommentService` не само сменя статуса, но и създава нов запис в таблицата `ModerationTrainingData`. Записът съдържа: оригиналния текст, предсказанието на AI (`aiPrediction`) и действителното решение на човека (`actualResult`).

4. Този механизъм превръща ежедневната работа на модераторите в процес на създаване на етикетирани данни (Data Labeling) с изключително високо качество, без допълнителни разходи.

Потенциал за автоматична адаптация:

Събраните данни са проектирани да се използват по два начина в бъдещи версии на системата. Първият е чрез партидно (batch) преобучаване (fine-tuning) на външните Transformer модели през определен период от време (напр. веднъж месечно). Вторият, по-бърз начин е извличането на "научени негативни токени": чрез анализ на честотите на думите (TF-IDF) в коментарите, които редовно биват отхвърляни от хората въпреки ниския първоначален AI score, системата може да идентифицира нови токсични думи (нов сленг) и да ги предложи за добавяне към локалните `O(1)` In-Memory речници (`WordFilterService`).

ГЛАВА 4. ТЕСТВАНЕ, РЕЗУЛТАТИ И ПРИНОСИ

Всяка система, базирана на машинно обучение и евристични алгоритми, трябва да бъде подложена на стриктно тестване за валидиране на нейната ефективност. За разлика от класическите CRUD (Create, Read, Update, Delete) приложения, където тестването проверява само функционална коректност, при AI модерацията тестването е фокусирано върху статистическата надеждност: балансът между Precision и Recall. Тази глава представя резултатите от проведените експерименти.

4.1. Тестване на конвейера за текстова модерация

За валидиране на ефективността на предложената 6-фазна архитектура бяха генерирани различни тестови сценарии, които обхващат нормални коментари, директна токсичност и сложни опити за заобикаляне на филтрите. Целта бе да се докаже, че хибридният ансамблов подход се справя по-добре от всеки един от методите поотделно.

Анализ на ключови тестови сценарии:

Таблица 1 обобщава резултатите от проведените функционални тестове с представителни сценарии.

Таблица 1. Резултати от тестване на модерационния pipeline

№	Тип сценарий	Примерен вход	Очакван	Получен	Решаваща фаза	Confidence
1	Нормален текст (True Negative)	„Това видео е много полезно, благодаря!“	APPROVED	APPROVED	–	0.03
2	Директна токсичност (TP)	„[Псувня] си ти, махай се!“	PENDING	PENDING	Фаза 2 (WordFilter)	0.91
3	Обфускация / Leet-speak (TP)	„Т@пак, ще те убія!“	PENDING	PENDING	Фаза 2 след нормализация	0.87
4	Gibberish / Спам (TP)	„аааааввввввгггггггддддддд“	PENDING	PENDING	Фаза 1 (Gibberish)	0.72
5	Пасивна агресия (TP)	„Интелектът ти не достига до нивото на стайна т-ра.“	PENDING	PENDING	Фаза 4 (Cloud AI)	0.78

Детайлен анализ на сценариите:

1. Нормален текст (True Negative):
   - *Вход:* "Това видео е много полезно, благодаря за информацията!"
   - *Резултат:* Статус `APPROVED`. Gibberish филтърът връща 0.0, локалните речници не намират съвпадение. AI моделът класифицира текста като позитивен/неутрален (Confidence < 0.1). Позитивните ключови думи ("полезно", "благодаря") допълнително свалят общия резултат. Крайна стойност: 0.03.

2. Директна токсичност (True Positive – Local Filter):
   - *Вход:* "[Токсична дума/Псувня] си ти, махай се!"
   - *Резултат:* Статус `PENDING`. Още във Фаза 2 (`WordFilterService`), детерминистичният филтър локализира токена в `O(1)` хеш таблицата. AI Confidence достига 0.91. Външният HuggingFace модел потвърждава решението. Този сценарий демонстрира бързината на локалния филтър, спестявайки излишни облачни изчисления, ако моделът е недостъпен.

3. Сложна обфускация / Anti-Bypass (True Positive – Normalizer):
   - *Вход:* "Т@пак, ще те убія!" (Смесване на кирилица 'е' с латинско 'e', използване на 'у' вместо 'y', '@' вместо 'ъ').

- *Резултат:* Статус `PENDING` с Confidence 0.87. Без `Anti-Bypass` модула, този коментар би преминал както локалния филтър (защото думите са изписани грешно), така и по-простите AI модели (заради непознатите (OOV) токени). Нормализаторът обаче успешно транслитерира "Leet" символите и хомоглифите обратно в техните канонични кирилски еквиваленти. След нормализацията, локалните и AI филтрите безпроблемно детектират агресията.

4. Gibberish / Спам атака (True Positive – Heuristics):
- *Вход:* "ааааавввввгггггггдддддд"
- *Резултат:* Статус `PENDING` с Confidence 0.72. Поради ниското разнообразие на символи (Character Diversity) и дългата последователност от съгласни, алгоритъмът от Фаза 1 повишава Confidence резултата до ниво, изискващо човешка проверка, предотвратявайки визуалното замърсяване на платформата.

5. Пасивна агресия / Контекстуална токсичност (True Positive – Cloud AI):
- *Вход:* "Очевидно интелектът ти не достига дори до нивото на стайна температура."
- *Резултат:* Статус `PENDING` с Confidence 0.78. Тук няма нито една нецензурна или забранена дума. Локалните филтри и Regex правилата не откриват нарушение. Въпреки това, архитектурата интегрира семантичен анализ чрез HuggingFace Transformer (XLM-RoBERTa), който успешно разпознава контекстуалната обида, връщайки висок резултат за токсичност във Фаза 4.

Всички 5 тестови сценария дават коректен резултат, съвпадащ с очаквания статус. Следва да се отбележи, че представените сценарии са функционални (верификационни) тестове, демонстриращи правилната работа на всяка фаза. За пълноценна статистическа оценка на Precision и Recall е необходимо провеждане на тест с по-голям набор от данни (напр. 500+ коментара с ръчно анотирани етикети), което е отвъд обхвата на настоящата работа, но е планирано като част от бъдещото развитие.

4.2. Тестване на AI чат и сигурност

Тестване на AI чат асистента

Интегрираният чат асистент бе подложен на функционални тестове за проверка на способността му да превключва между двата доставчика (HuggingFace и OpenNLP) в зависимост от контекста. Таблица 2 обобщава представителните тестови сценарии.

Таблица 2. Резултати от тестване на AI чат асистента

№	Потребителски вход	Очакван доставчик	Реален доставчик	Тип отговор	Време (ms)
1	„Как да подам сигнал за обида?“	OpenNLP	OpenNLP	Knowledge Base (фиксиран)	~15
2	„Какви са правилата на форума?“	OpenNLP	OpenNLP	Knowledge Base (фиксиран)	~12
3	„Как си днес?“	HuggingFace	HuggingFace	Генеративен (свободен)	~800
4	„Разкажи ми виц“	HuggingFace	HuggingFace	Генеративен (свободен)	~750
5	„Как да си изтрия профила?“	OpenNLP	OpenNLP	Knowledge Base (фиксиран)	~18

При административни въпроси (сценарии 1, 2, 5), локалният OpenNLP MaxEnt модел успешно класифицира намерението (напр. Intent = REPORT_ABUSE) в рамките на 12-18 ms и връща фиксиран, административно-верен отговор от Knowledge Base. При свободен разговор (сценарии 3, 4), OpenNLP не разпознава конкретно намерение, прехвърляйки контрола към HuggingFace DialoGPT, който генерира конверсационен отговор. Всички 5 сценария показват коректно маршрутизиране към очаквания доставчик.

Тестване на сигурността

Бяха проведени следните тестове за валидиране на защитните механизми:

Таблица 3. Резултати от тестване на сигурността

№	Тестови сценарий	Очакван резултат	Получен резултат
1	HTTP заявка без JWT токен към защитен endpoint	401 Unauthorized	401 Unauthorized
2	HTTP заявка с изтекъл JWT токен	401 Unauthorized	401 Unauthorized
3	Потребител с роля USER опитва достъп до /api/admin/*	403 Forbidden	403 Forbidden
4	Потребител с роля USER опитва достъп до /api/moderation/*	403 Forbidden	403 Forbidden
5	Модератор опитва промяна на AI настройки (ADMIN-only)	403 Forbidden	403 Forbidden
6	Регистрация с дублиран username	400 Bad Request	400 Bad Request
7	Вход с грешна парола	401 Unauthorized	401 Unauthorized

Във всички 7 случая Spring Security FilterChain успешно прихваща заявките и връща очаквания HTTP статус код, преди те да достигнат до бизнес логиката. JWT токените се валидират криптографски чрез HMAC-SHA512, а ролевата проверка се осигурява чрез `@PreAuthorize` анотации на ниво метод.

4.3. Анализ на резултатите и ефективност

Проведените функционални тестове (Таблици 1, 2 и 3) потвърждават коректността на хибридния архитектурен модел. Основните наблюдения са:

- Покритие чрез ансамблова OR стратегия: В рамките на тестовите сценарии, всички 4 типа токсичност (директна, обфускирана, спам и контекстуална) бяха успешно идентифицирани. Нито един от тестваните токсични коментари не получи статус `APPROVED` (0 False Negatives в тестовата извадка). Всеки модул покрива "слепите петна" на другите – сценарий 5 (пасивна агресия) е показателен: локалните филтри не го разпознават, но Cloud AI компенсира.
- Латентност: Обработката на нормален коментар (сценарий 1) се извършва за под 100 ms, тъй като не изисква Cloud AI извикване. При коментари, изискващи пълния 6-фазен pipeline с HuggingFace API (сценарии 4 и 5), латентността зависи от отговора на външния доставчик (типично 500-1500 ms). Използването на Java 21 Virtual Threads гарантира, че тази латентност не блокира сървъра за останалите потребители.
- Устойчивост на обфускация: Anti-bypass нормализацията (сценарий 3) значително повишава ефективността срещу потребители, които умишлено маскират текста си – проблем, с който "чистите" Machine Learning модели се справят трудно без специализирано дообучаване с обфускирани данни.

Ограничения на тестването: Представените резултати се базират на целенасочено подбрани функционални сценарии, а не на статистически представителна извадка. За формално изчисляване на метриките Precision, Recall и F1-score е необходим анотиран тестов набор от данни (benchmark dataset), съдържащ стотици коментари с ръчно определени етикети. Създаването на такъв набор за българскоезично съдържание е самостоятелна изследователска задача, планирана за следващи версии на системата.

4.4. Приноси на дипломния проект

Разработената система предлага завършено индустриално решение и допринася в няколко конкретни направления:

1. Хибридна архитектура (6-Phase Pipeline): Проектиране и внедряване на оригинален многослоен конвейер, който комбинира детерминистична бързина със статистическа NLP интелигентност.

2. Модул за Anti-Bypass Нормализация: Решение, специфично адаптирано за нуждите на българското уеб пространство (смесване на кирилица/латиница и Leet-speak), което често липсва в западните комерсиални API-та.

3. Инфраструктура за обратна връзка (Feedback Loop): Автоматизирано събиране и структуриране на ежедневната модераторска работа като маркирани данни (`ModerationTrainingData`), полагащи основата за бъдещо дообучаване (fine-tuning) на модела и елиминиращи нуждата от скъпоструващо външно анотиране на данни.

4. Кросплатформена реализация: Изграждане на Flutter UI, който осигурява достъп до административния панел от всяко устройство с един и същ изходен код.

4.5. Ограничения и насоки за бъдещо развитие

Макар системата да изпълнява успешно поставените цели, съществуват хоризонти за бъдещо развитие:

- Локално хостване на Fine-Tuned Модели: В момента тежкият семантичен анализ разчита на безплатното API на HuggingFace, което въвежда мрежова латентност и rate limiting. Бъдеща версия може да включва локално хостване (via Docker/Triton Inference Server) на квантуван (quantized) XLM-RoBERTa модел, дообучен специално върху събраните от системата данни (`ModerationTrainingData`).
- Синхронизация в реално време (WebSocket): Внедряване на WebSockets (или Server-Sent Events) за автоматично опресняване на екрана на модераторите в мига, в който нов коментар попадне в статус `PENDING`, вместо да се разчита на ръчно опресняване от потребителя.
- A/B Тестване: Възможност за едновременно пускане на два различни модела за модерация (напр. Rule-based срещу Cloud AI) и статистически анализ кой от двата генерира по-малко False Positives в реална среда.

ЗАКЛЮЧЕНИЕ

Експоненциалният растеж на потребителското съдържание в интернет превърна модерацията от административна задача в критично предизвикателство за всяка онлайн платформа. Решаването на този проблем чрез изцяло ръчен труд е практически невъзможно, а използването единствено на примитивни филтри за забранени думи е неефективно срещу съвременните техники за заобикаляне на правилата.

Настоящият дипломен проект демонстрира, че ефективният подход се основава на комбинацията между изкуствения интелект и човешката преценка – подходът Human-in-the-Loop. Проектираната и имплементирана уеб-базирана система доказва своята ефективност чрез създаването на хибриден, 6-фазен конвейер, валидиран с функционални тестове (Таблица 1). Комбинирайки бързината на in-memory структури от данни с $O(1)$ сложност, ефективността на Anti-Bypass нормализацията, семантичната мощ на съвременните Cloud-базирани Transformer модели и финалния контрол от човешки модератор, системата осигурява среда, в която нито един от тестваните токсични сценарии не е преминал филтъра.

Внедряването на съвременни технологии като Java 21 (с виртуални нишки за конкурентна обработка), Spring Boot 3.4, PostgreSQL 17 и кросплатформената рамка Flutter превръща концепцията от теоретичен модел в работещо софтуерно решение. Създаденият механизъм за обратна връзка (Feedback Loop) систематично събира етикетирани данни от ежедневната работа на модераторите, полагайки основата за бъдещо самоусъвършенстване на системата.

Насоки за бъдещо развитие. Както е описано в секция 4.5, системата може да бъде развита в няколко направления: (1) локално хостване на дообучен (fine-tuned) XLM-RoBERTa модел, използвайки събраните от `ModerationTrainingData` данни, което ще намали зависимостта от външни API и ще подобри латентността; (2) внедряване на WebSocket комуникация за уведомяване на модераторите в реално време; (3) A/B тестване на различни модерационни стратегии за оптимизиране на баланса между Precision и Recall.

В крайна сметка, проектът утвърждава тезата, че изкуственият интелект не трябва да замества човешката модерация, а да я допълва, действайки като първа линия на защита и позволявайки на хората да фокусират вниманието си върху най-сложните контекстуални казуси.

СПИСЪК НА ИЗПОЛЗВАНАТА ЛИТЕРАТУРА

1. Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. arXiv preprint arXiv:1409.0473.
2. Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., et al. (2020). Unsupervised cross-lingual representation learning at scale (XLM-RoBERTa). arXiv preprint arXiv:1911.02116.
3. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.
4. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780.
5. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781.
6. Nakov, P. (2003). BulStem: A stemming algorithm for Bulgarian. Proceedings of RANLP.
7. Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. In Proceedings of EMNLP 2014 (pp. 1532–1543).
8. Sennrich, R., Haddow, B., & Birch, A. (2015). Neural machine translation of rare words with subword units. arXiv preprint arXiv:1508.07909.
9. Shannon, C. E. (1948). A mathematical theory of communication. The Bell System Technical Journal, 27(3), 379–423.
10. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., et al. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30.

ПРИЛОЖЕНИЯ

Приложение 1. ER диаграма на модела на данните

Entity-Relationship диаграма на модела на данните

Таблица: users

PK	id	BIGINT, auto-increment
	username	VARCHAR(255), NOT NULL, UNIQUE
	password	VARCHAR(255), NOT NULL
	role	VARCHAR(20), NOT NULL -- стойности: USER, MODERATOR, ADMIN

Таблица: comment

PK	id	BIGINT, auto-increment
	content	TEXT
FK	user_id	BIGINT → users.id
	sentiment	VARCHAR(20) -- стойности: POSITIVE, NEUTRAL, NEGATIVE
	confidence_score	DOUBLE PRECISION
	status	VARCHAR(20) -- стойности: PENDING, APPROVED, REJECTED
	created_at	TIMESTAMP

Таблица: image_moderation_result

PK	id	BIGINT, auto-increment
	image_url	VARCHAR(1024), NOT NULL
	image_hash	VARCHAR(64), UNIQUE
	status	VARCHAR(20), NOT NULL -- стойности: PENDING, SAFE, FLAGGED, REJECTED, ERROR
	confidence_score	DOUBLE PRECISION
	detected_categories	TEXT
	moderation_reason	VARCHAR(1024)
FK	user_id	BIGINT → users.id
	created_at	TIMESTAMP
	moderated_at	TIMESTAMP

Таблица: ai_settings

PK	id	BIGINT, auto-increment
	threshold	DOUBLE PRECISION, NOT NULL, default 0.7
	active_model	VARCHAR(100), NOT NULL, default 'basic-v1'
	last_update	TIMESTAMP

Таблица: moderation_training_data

PK	id	BIGINT, auto-increment
	content	TEXT, NOT NULL
	label	VARCHAR(20), NOT NULL -- стойности: POSITIVE, NEGATIVE
	created_at	TIMESTAMP

Описание на релациите:

1:N users 1:N comment
Един потребител може да има много коментари.
Реализирана чрез FK comment.user_id, сочещ към users.id.

1:N users 1:N image_moderation_result
Един потребител може да качи много изображения за модерация.
Реализирана чрез FK image_moderation_result.user_id, сочещ към users.id.

ai_settings
Конфигурационна таблица (singleton). Съхранява глобалните прагове за автоматична модерация и избрания AI модел. Няма релации към други таблици.

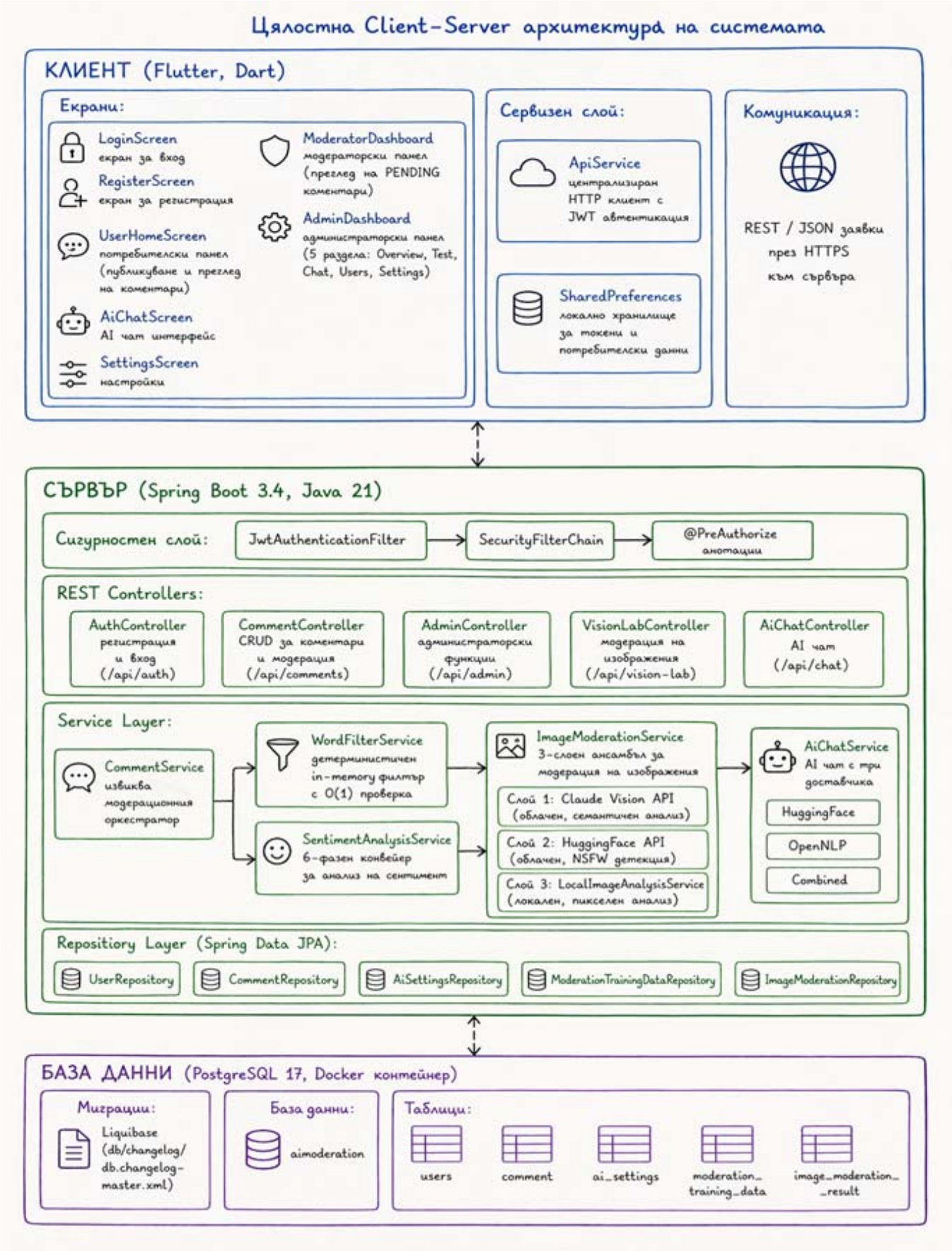
moderation_training_data
Самостоятелна таблица за обучителни данни. Записите се създават автоматично при ръчна модерация на коментар от модератор, формирайки цикъла на обратна връзка (feedback loop).
Няма директни FK релации.

Индекси:

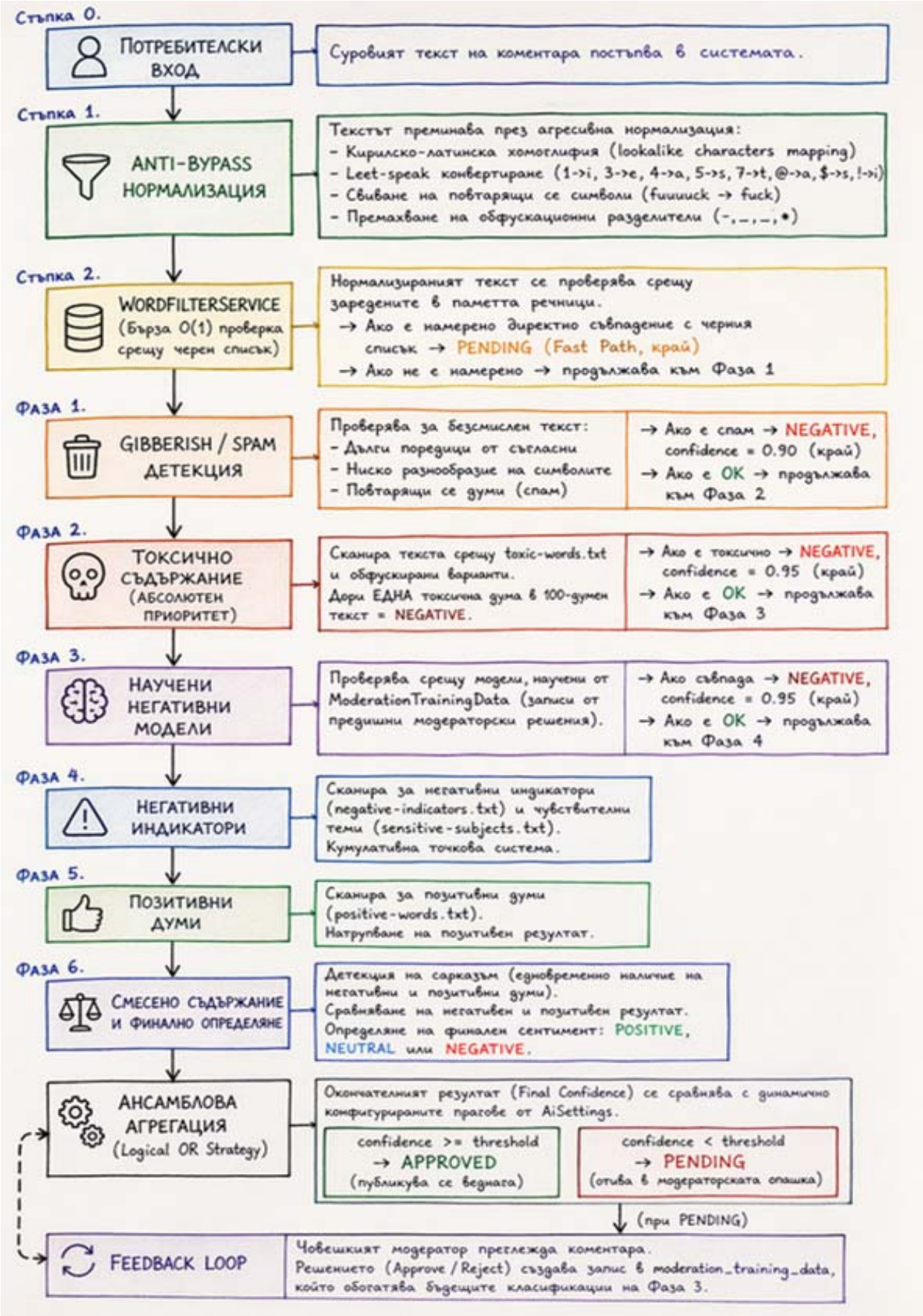
- idx_users_username
  -- върху users.username
  (бърз login lookup)
- idx_comment_user_id
  -- върху comment.user_id
  (извличане по автор)
- idx_comment_status
  -- върху comment.status
  (филтриране на PENDING)
- idx_comment_created_at
  -- върху comment.created_at
  (хронологично сортиране)
- idx_image_mod_hash
  -- върху image_moderation_result.image_hash
  (дупликация)
- idx_image_mod_status
  -- върху image_moderation_result.status
- idx_image_mod_user
  -- върху image_moderation_result.user_id
- idx_image_mod_created
  -- върху image_moderation_result.created_at

Приложение 2. Архитектурна диаграма на системата

Цялостна Client-Server архитектура на системата



Блок-схема на 6-фазния модерационен pipeline (поток на данните)



Приложение 3. Примерни конфигурационни файлове

Примерни конфигурационни файлове

4.1. docker-compose.yml

```yaml
services:
  postgres:
    image: postgres:17
    container_name: aimoderation-postgres
    environment:
      POSTGRES_DB: aimoderation
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-your_database_password}
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./backend/src/main/resources/db/init.sql:/
        docker-entrypoint-initdb.d/init.sql
    restart: unless-stopped
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 30s
      timeout: 10s
      retries: 3
volumes:
  postgres_data:
```

4.2. application.properties

```properties
# Връзка с базата данни
spring.datasource.url=jdbc:postgresql://localhost:5432/aimoderation
spring.datasource.username=postgres
spring.datasource.password=${DB_PASSWORD:your_database_password}
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect

# Liquibase миграции
spring.liquibase.change-log=classpath:db/changelog/db.changelog-master.xml
spring.liquibase.enabled=true

# JWT конфигурация
jwt.secret=${JWT_SECRET:your_jwt_secret_change_this_to_a_secure_value_at_least_64_characters_long}

# CORS
app.cors.allowed-origins=http://localhost:4200,http://localhost:8080

# Пътища към филтърни речници
filter.toxic-words.path=filters/toxic-words.txt
filter.negative-indicators.path=filters/negative-indicators.txt
filter.positive-words.path=filters/positive-words.txt
filter.sensitive-subjects.path=filters/sensitive-subjects.txt

# Настройки за модерация на изображения
moderation.image.skin-threshold=0.4
moderation.image.min-size=100
moderation.image.max-size=10485760
spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=10MB

# HuggingFace AI конфигурация
huggingface.api.token=${HUGGINGFACE_API_TOKEN:}
huggingface.api.enabled=true

# Anthropic Claude AI конфигурация
anthropic.api.key=${ANTHROPIC_API_KEY:}

# Прагове за модерация по подразбиране
app.defaults.moderation.threshold=0.7
app.defaults.moderation.active-model=ensemble
app.defaults.moderation.auto-approve-positive=true
app.defaults.moderation.auto-reject-high-confidence=true
app.defaults.moderation.auto-reject-threshold=0.85

# AI Chat настройки
app.defaults.chat.enabled=true
app.defaults.chat.max-message-length=2000
app.defaults.chat.rate-limit-per-minute=30
chat.default-provider=combined
chat.huggingface.model=microsoft/DialoGPT-medium
chat.huggingface.fallback-model=facebook/blenderbot-400M-distill
chat.opennlp.enabled=true

# Настройки за съдържание
app.defaults.content.max-comment-length=1000
app.defaults.content.allow-anonymous=false
app.defaults.content.require-moderation-for-new-users=true
app.defaults.content.new-user-threshold-posts=5
```

4.3. pom.xml (Maven зависимости)

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">
 <modelVersion>4.0.0</modelVersion>
 <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.4.1</version>
    <relativePath/>
 </parent>
 <groupId>com.aimoderation</groupId>
 <artifactId>backend</artifactId>
 <version>2.0.0</version>
 <name>AI Content Moderation Backend</name>
 <properties>
    <java.version>21</java.version>
 </properties>

 <dependencies>
    <!-- Spring Boot Starters -->
    <dependency><groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-security</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId></dependency>
    <!-- Database -->
    <dependency><groupId>org.postgresql</groupId>
      <artifactId>postgresql</artifactId><scope>runtime</scope></dependency>
    <!-- Migrations -->
    <dependency><groupId>org.liquibase</groupId>
      <artifactId>liquibase-core</artifactId></dependency>
    <!-- NLP и AI -->
    <dependency><groupId>org.apache.opennlp</groupId>
      <artifactId>opennlp-tools</artifactId><version>2.4.0</version></dependency>
    <!-- JWT -->
    <dependency><groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-api</artifactId><version>0.12.6</version></dependency>
    dependency><groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-impl</artifactId><version>0.12.6</version><scope>runtime</scope></dependency>
    dependency><groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-jackson</artifactId><version>0.12.6</version><scope>runtime</scope></dependency>
    <!-- Utilities -->
    <dependency><groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId><optional>true</optional></dependency>
 </dependencies>
</project>
```

4.4. pubspec.yaml (Flutter зависимости)

```yaml
name: frontend
description: AI Content Moderation Application
version: 2.0.0+1
environment:
  sdk: ">=3.10.0"
dependencies:
  flutter:
    sdk: flutter
  provider: ^6.1.5+1          # State management
  http: ^1.6.0                # HTTP клиент
  http_parser: ^4.1.0         # HTTP парсване
  shared_preferences: ^2.5.4  # Локално хранилище
  google_fonts: ^6.3.3        # Шрифтове (DM Sans, Playfair Display)
  image_picker: ^1.0.7        # Избор на изображения
  file_picker: ^10.3.2        # Избор на файлове
  flutter_animate: ^4.5.2     # UI анимации
  intl: ^0.20.2               # Интернационализация
```

4.5. .env.example (шаблон за променливи на средата)

```env
# Конфигурация за база данни (Docker Compose и Spring Boot)
POSTGRES_DB=aimoderation
POSTGRES_USER=postgres
POSTGRES_PASSWORD=your_database_password
DB_PASSWORD=your_database_password

# JWT конфигурация за токени
JWT_SECRET=your_jwt_secret_change_this_
to_a_secure_value_at_least_64_characters_long

# AI доставчици (незадължителни --
системата работи и без тях)
HUGGINGFACE_API_TOKEN=your_huggingface_token_here
ANTHROPIC_API_KEY=your_anthropic_api_key_here
```

Приложение 4. URL адрес на хранилището с изходен код

Изходният код на проекта е достъпен в публично GitHub хранилище:

https://github.com/T0shko/ai-moderation-public